Lab Experiment – 10

A Search Algorithm

Aim

To implement the A* search algorithm in Python to find the shortest path from a start node to a goal node.

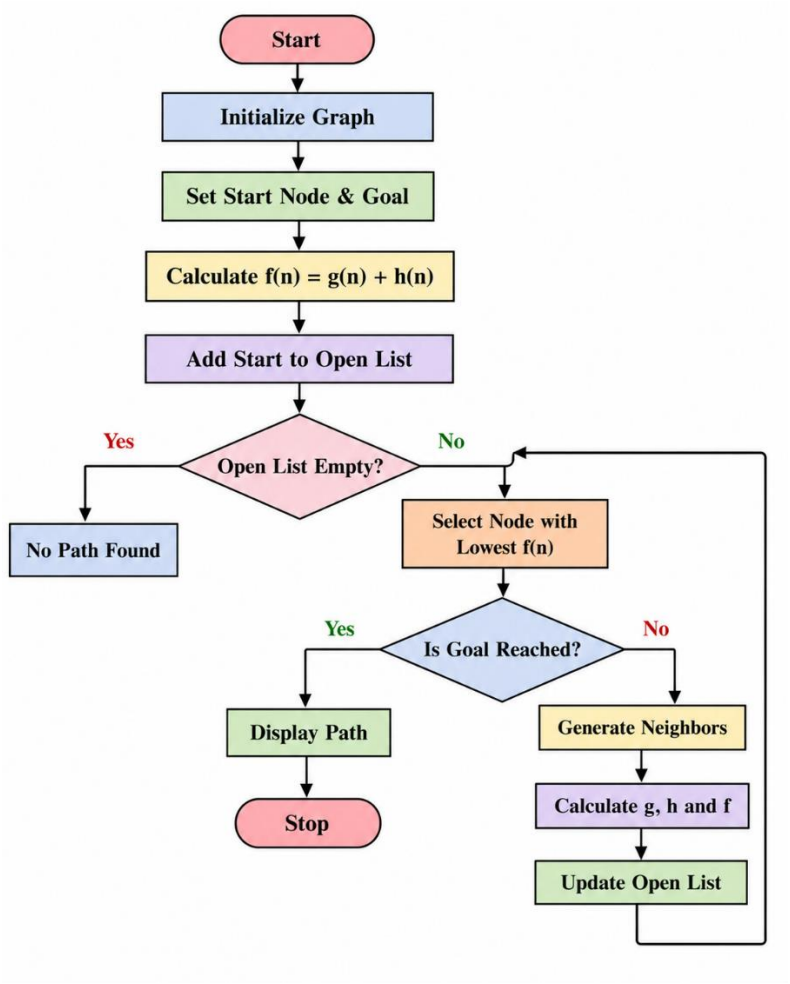
Objective

To find an optimal path between two nodes by using the actual path cost and a heuristic estimate.

Algorithm

1. Start with the initial node.
2. Set the cost of the start node as `0`.
3. Calculate the evaluation function:
   f(n) = g(n) + h(n)
   - `g(n)` = actual cost from the start node to node `n`.
   - `h(n)` = estimated cost from node `n` to the goal.
4. Add the start node to the priority queue.
5. Select the node having the lowest `f(n)` value.
6. If the selected node is the goal node, display the path.
7. Otherwise, generate its neighboring nodes.
8. Calculate the cost for each neighbor.
9. Add or update the neighbors in the priority queue.
10. Repeat until the goal is reached.

FlowChart



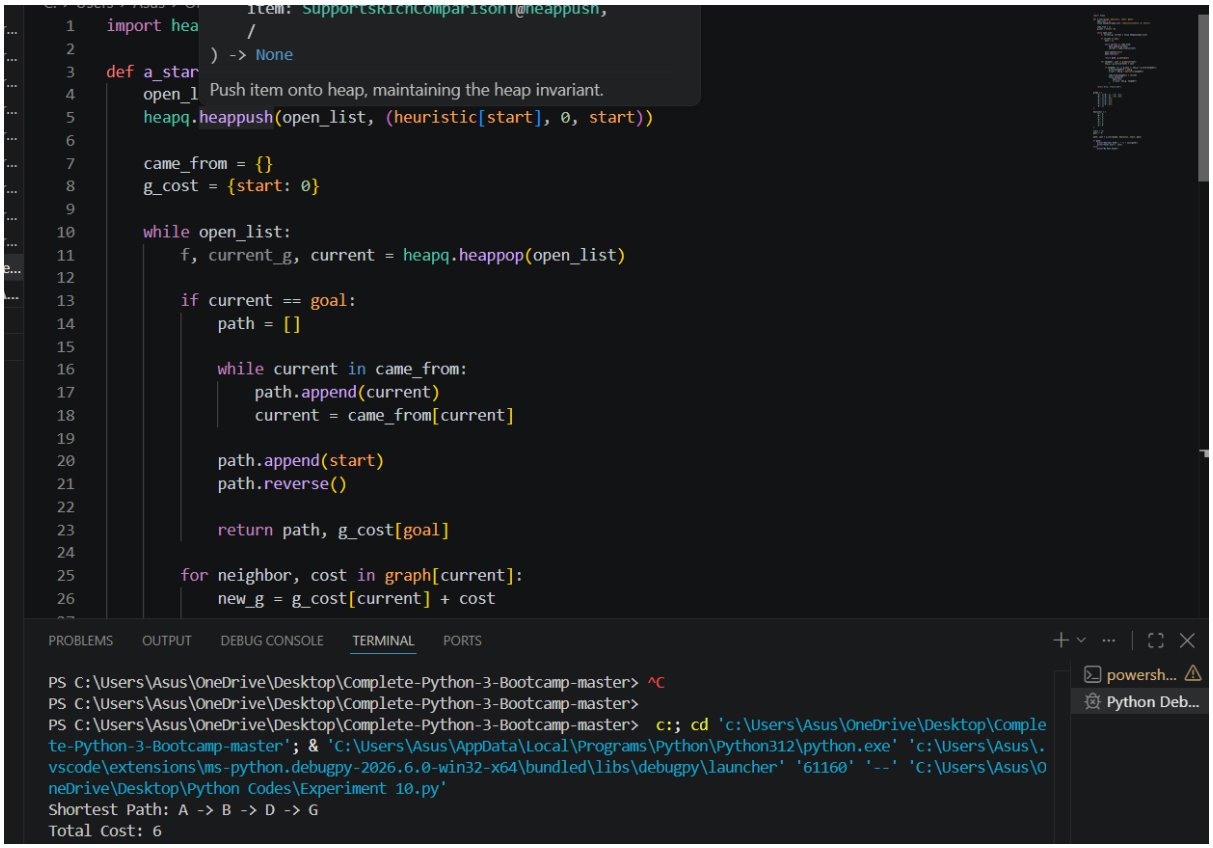
Code

```
import heapq


def a_star(graph, heuristic, start, goal):
    open_list = []
    heapq.heappush(open_list, (heuristic[start], 0, start))


    came_from = {}
    g_cost = {start: 0}


    while open_list:
        f, current_g, current = heapq.heappop(open_list)
```

```
        if current == goal:
            path = []

            while current in came_from:
                path.append(current)
                current = came_from[current]

            path.append(start)
            path.reverse()

            return path, g_cost[goal]

        for neighbor, cost in graph[current]:
            new_g = g_cost[current] + cost

            if neighbor not in g_cost or new_g < g_cost[neighbor]:
                g_cost[neighbor] = new_g
                f_cost = new_g + heuristic[neighbor]

                came_from[neighbor] = current
                heapq.heappush(
                    open_list,
                    (f_cost, new_g, neighbor)
                )

    return None, float('inf')



graph = {
    'A': [('B', 1), ('C', 4)],
    'B': [('D', 2), ('E', 5)],
    'C': [('F', 3)],
    'D': [('G', 3)],
```

```
    'E': [('G', 1)],
    'F': [('G', 2)],
    'G': []
}

heuristic = {
    'A': 6,
    'B': 5,
    'C': 4,
    'D': 3,
    'E': 1,
    'F': 2,
    'G': 0
}

start = 'A'
goal = 'G'

path, cost = a_star(graph, heuristic, start, goal)

if path:
    print("Shortest Path:", " -> ".join(path))
    print("Total Cost:", cost)
else:
    print("No Path Found")
```

OUTPUT



Result

The A* search algorithm was successfully implemented in Python and the shortest path from the start node to the goal node was obtained.

Conclusion

The A* algorithm finds an optimal path by combining the actual cost `g(n)` and heuristic cost `h(n)` using **f(n) = g(n) + h(n)**. It is widely used in pathfinding and navigation problems.